

ECE 167 Lab 4

Demetrius Rauck

March 3, 2024

1 Introduction

In this lab we were tasked with attitude estimation using sensor fusion to track attitude using a direction cosine matrix. We familiarized ourselves with kinematic equations for frames and the matlab code that provided the functions we'll use.

Git commit: f54ce58a2ff3d212722447f61dc28dba97e0314c

2 DCM to Euler Angles

To gain the euler angles, angles that determine an objects orientation about the z, y and x axes sequentially, we wrote a function that extracts them from a DCM. The DCM represents the rotations across the axis sequentially, with every proceeding rotation being multiplied by the objects current rotation matrix. The convention we were using was a rotation across the z axis by Ψ , then across the y axis by Θ and finally across the x axis by Φ .

The angles Ψ , Θ , and Φ can be extracted from the DCM with inverse trig and relating the simpler matrix terms to one another, canceling out terms to get a single angle. Some of these terms can make the euler angles undefined however, due to gimbal lock singularity. The relation of trig terms can make it such that it can be out of bounds for the inverse sin function or dividing by zero in the inverse tangent function.

To extract the euler angles from the DCM, we wrote a function that first checked for a potential gimbal lock singularity of certain terms in the matrix. It then did the inverse trig using terms of the DCM and then converted each of the angles into degrees.

```
eulerAngles DCMtoEuler(float dcm[3][3]){
    eulerAngles angles;

    float roll_RAD;
    float pitch_RAD;
    float yaw_RAD;
```

```

    if(abs(dcm[0][2]) == 1 || dcm[0][0] == 0 || dcm[2][2] == 0){
        printf("Gimble lock Singularity\n");
    } else{
        //X, roll, phi
        roll_RAD = atan(dcm[1][2]/dcm[2][2]);
        angles.roll = roll_RAD * 180 / M_PI; //degrees

        //Y, pitch, theta
        pitch_RAD = -asin(dcm[0][2]);
        angles.pitch = pitch_RAD * 180 / M_PI; //degrees

        //Z, yaw, psi
        yaw_RAD = atan(dcm[0][1]/dcm[0][0]);
        angles.yaw = yaw_RAD * 180 / M_PI; //degrees
    }

    return angles;
}

```

To test this we entered in an identity matrix which derived euler angles of 0. We then hardcoded angles and provided a general form rotation matrix. Since we hardcoded the angles, the output angles would be the exact same as the input angles.

```

//PART 2
#ifdef PART_2
int main(void){
    BOARD_Init();

    //Identity matrix
    //float testMatrix[3][3] = {{1,0,0},{0,1,0},{0,0,1}};

    //GIMBLE LOCK
    //float testMatrix[3][3] = {{0,0,0},{0,0,0},{0,0,0}};

    //TEST 3
    float theta = M_PI/4;
    float psi = M_PI/4;
    float phi = M_PI/4;

    float testMatrix[3][3] =
    {{cos(theta)*cos(psi), cos(theta)*sin(psi), -sin(theta)},
    {cos(psi)*sin(theta)*sin(phi) - sin(psi)*cos(phi),
    sin(psi)*sin(theta)*sin(phi) + cos(psi)*cos(phi),
    cos(theta)*sin(phi)},

```

```

    {cos(psi)*sin(theta)*cos(phi) + sin(psi)*sin(phi),
    sin(psi)*sin(theta)*cos(phi) - cos(psi)*sin(phi),
    cos(theta)*cos(phi)}};

    eulerAngles testResult;

    testResult = DCMtoEuler(testMatrix);

    printf("Roll: %f\n", testResult.roll);
    printf("Pitch: %f\n", testResult.pitch);
    printf("Yaw: %f\n", testResult.yaw);

    return 0;
}

```

3 Integration of Constant Body-Fixed Rates

To track attitude, we used attitude propagation to integrate rotation from a known starting point.

To start the C function, we used hard coded angles and assembled a DCM into an array with the required functions and equations. The part that was the most complex was doing matrix math in C. Functions couldn't return arrays so had to be supplied with answer arrays, meaning many functions couldn't be used as an assignment and as an argument for a function at the same time, but would have to be assigned first and then use the answer variable as an argument. Adding and subtracting matrices to values or other matrices was a simple process of a nested for loop going element by element and doing arithmetic. It was more complex to do matrix multiplication as it required 3 nested for loops to do the proper linear algebra with the possibility of matrices of different sizes.

```

void multiplyMatrices(float firstMatrix[SIZE][SIZE],
float secondMatrix[SIZE][SIZE], float result[SIZE][SIZE]) {
    int i, j, k;

    // Multiply firstMatrix and secondMatrix, store in result
    for(i = 0; i < SIZE; i++) {
        for(j = 0; j < SIZE; j++) {
            result[i][j] = 0;
            for(k = 0; k < SIZE; k++) {
                result[i][j] += firstMatrix[i][k] * secondMatrix[k][j];
            }
        }
    }
}

```

3.1 Simple Forward Integration

To derive the new rotation matrix we coded the formula $R_{k+1} = R_k - [\omega x] R_k \Delta T$ in C. $[\omega x]$ is the cross product matrix of the vector $[p, q, r]^T$ which represent the rates of rotation about the x, y and z axes.

The order of operations in the code was to first set up $[\omega x]$ based on the inputs p, q and r and then matrix multiply it by the input DCM R_k . Then we scalar multiply it element by element by ΔT then subtract the resulting matrix from R_k element by element. This got us our resultant R_{k+1} .

```
void attitudePropagation(float r, float p, float q, float Rk[SIZE][SIZE], float dT, float Rk1[SIZE][SIZE])
{
    float omegaCross[3][3] = {{0,-r,q},{r,0,-p},{-q,p,0}};
    float Rdot[3][3];

    multiplyMatrices(omegaCross, Rk, Rdot);
    fowardIntegration(Rk, Rdot, dT, Rk1);
}

void fowardIntegration(float Rk[SIZE][SIZE], float Rdot[SIZE][SIZE], int dT, float Rk1[SIZE][SIZE])
{
    int i, j;
    for(i = 0; i < SIZE; i++) {
        for(j = 0; j < SIZE; j++) {
            Rk1[i][j] = Rk[i][j] - Rdot[i][j] * dT; //minus because omegaX is negative
        }
    }
}

//PART 3
#ifdef PART_31

int main(void){
    BOARD_Init();

    float theta, psi, phi;
    float r, p, q, dT;
    float Rk1[3][3];
    float ROrtho[3][3];

    //TEST VALUES
    theta = M_PI/4;
    psi = M_PI/4;
    phi = M_PI/4;

    r = 1;
    p = 1;
    q = 1;
}
```

```

dT = 1;

float testMatrix[3][3] =
{{cos(theta)*cos(psi), cos(theta)*sin(psi), -sin(theta)},
{cos(psi)*sin(theta)*sin(phi) - sin(psi)*cos(phi),
sin(psi)*sin(theta)*sin(phi) + cos(psi)*cos(phi),
cos(theta)*sin(phi)},
{cos(psi)*sin(theta)*cos(phi) + sin(psi)*sin(phi),
sin(psi)*sin(theta)*cos(phi) - cos(psi)*sin(phi),
cos(theta)*cos(phi)}};

attitudePropagation(r, p, q, testMatrix, dT, Rk1);
printMatrix(Rk1);

printf("\n");

//attitudeOrtho(Rk1, ROrtho);
//printMatrix(ROrtho);
}

#endif

```

We verified this function in our own matlab code which did the same procedure but already had built in functions to deal with matrices.

```

syms theta psi phi p r q dT
r = 1;
p = 1;
q = 1;
theta = pi/4;
psi = pi/4;
phi = pi/4;
dT = 1;
Rk = [ (cos(theta)*cos(psi)) (cos(theta)*sin(psi)) (-sin(theta)) ; (sin(phi)*sin(theta)*cos(theta)*sin(psi))
(0 -r q; r 0 -p; -q p 0];
omegacross = [0 -r q; r 0 -p; -q p 0];

Rk1 = Rk + ((-omegacross * Rk ) * dT)

```

We additionally attempted to reorthonormalize R_{k+1} with the formula $R_{\perp} = R_{k+1} + (1/2)[I - RR^T]R$ which worked in matlab but didn't function in C most likely due to the amount of complex operations.

3.2 Matrix Exponential Form

Deriving the Matrix Exponential Form consisted of coding the formula $R_{k+1} = e^{-[\omega x]\Delta T} R_k$. This equation consists of the same variables from Simple Forward

Integration, but instead will be used for an exponential form. The coding in C is based on the provided matlab code from Rexp.m. With this, a piece-wise function was created depending on how close wnorm would be to zero. When wnorm is zero, this will cause issues where it will give undefined values when later used to determine the final R_{k+1} .

```
void matrixExp(float r, float p, float q, float deltaT, float Rexp[SIZE][SIZE]){
    float sincW, oneMinusCosW;

    float omegaCross[3][3] = {{0,-r,q},{r,0,-p},{-q,p,0}};

    float wnorm = sqrt((r*r)+(q*q)+(p*p));

    if(wnorm < 0.2){ //might have to change number
        sincW = deltaT - (pow(deltaT, 3) * pow(wnorm, 2))/6.0 + (pow(deltaT, 5) * pow(wnorm, 2))/24.0;
        oneMinusCosW = (pow(deltaT, 2))/2.0 - (pow(deltaT, 4) * pow(wnorm, 2))/24.0 + (pow(deltaT, 6) * pow(wnorm, 2))/120.0;
    }else{
        sincW = sin(wnorm * deltaT)/ wnorm;
        oneMinusCosW = (1.0 - cos(wnorm * deltaT)) / pow(wnorm, 2);
    }

    //R_exp = [1 0 0;0 1 0;0 0 1] - sincW * wx + oneMinusCosW * wx * wx;
    printf("%f\n", sincW);
    printf("%f\n", oneMinusCosW);

    //wx*wx
    float matrixWXX[SIZE][SIZE];
    multiplyMatrices(omegaCross, omegaCross, matrixWXX);
    printMatrix(matrixWXX);
    printf("\n");

    //oneMinusCosW * wx * wx
    float matrixOneMinusCoswWXX[SIZE][SIZE];
    multiplyMatrixByScalar(matrixWXX, oneMinusCosW, matrixOneMinusCoswWXX);
    printMatrix(matrixOneMinusCoswWXX);
    printf("\n");

    //sincW * wx
    float matrixSincwWX[SIZE][SIZE];
    multiplyMatrixByScalar(omegaCross, sincW, matrixSincwWX);
    printMatrix(matrixSincwWX);
    printf("\n");

    //sincW * wx + oneMinusCosW * wx * wx
    float matrixAdded[SIZE][SIZE];
```

```

    addMatrices(matrixOneMinusCoswWXWX, matrixSincwWX, matrixAdded);
    printMatrix(matrixAdded);
    printf("\n");

    //[1 0 0;0 1 0;0 0 1] - sincW * wx + oneMinusCosW * wx * wx;
    subtractMatrices(ID_MATRIX, matrixAdded, Rexp);

}

```

Both the results from the matlab code and the C code were compared. They were not giving the same exact results, but this could be due to how both matlab and C handle numbers differently.

4 Open Loop Gyroscope Integration

4.1 Simulated Data

To test out the algorithm to derive the euler angles we used the matlab function provided to create a random trajectory. We then used a for loop to iterate through all values of the created trajectory. We initialized the first rotation matrix as the general form of the DCM with the input Ψ , Θ , and Φ from the first set of euler angles from the created trajectory. In the loop, we converted the vector of the gyroscope readings at that iteration to radians and divided by 131 to account for the board's error. We then use the IntegrateOpenLoop function using the matrix exponential technique, then derive the euler angles from that. I then repeated the experiment with noise in the random trajectory, then both with and without using the identity matrix as the initial rotation matrix, and finally adding a gyro bias, the mean of the first second of the gyro readings on each axis, and repeating each experiment.

The results showed that the most accurate algorithm was done when we had no noise, no noise with a gyro bias, and noise with gyro bias. This was interesting as the gyro bias usually ended up being zero as most of the gyro readings were zero that were intermittently interrupted by a change, meaning there was no difference between the gyro bias and normal. The random trajectory function being used for each experiment would explain this as the functions that were most accurate varied in their accuracy.

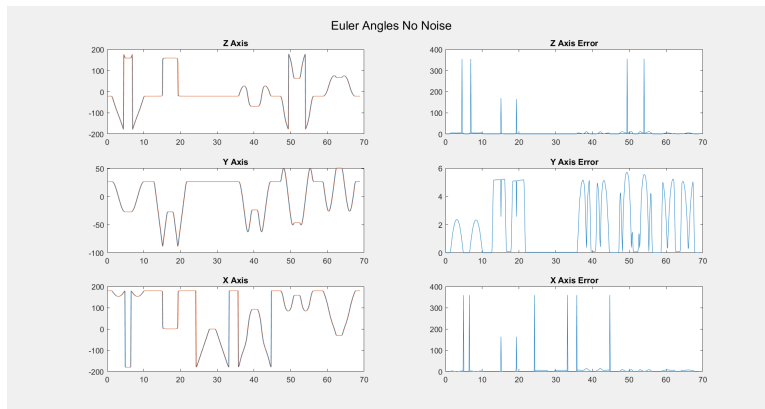


Figure 1: Euler Angles No Noise

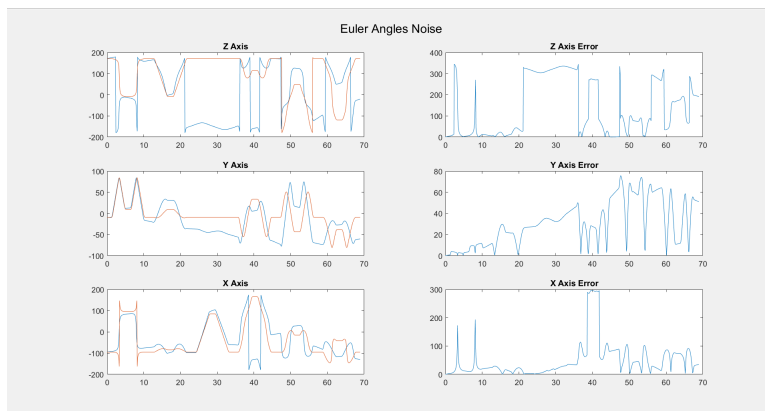


Figure 2: Euler Angles Noise

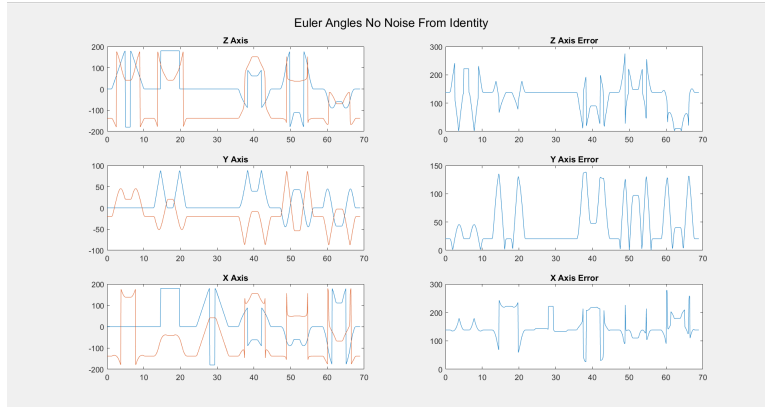


Figure 3: Euler Angles No Noise From Identity



Figure 4: Euler Angles Noise From Identity

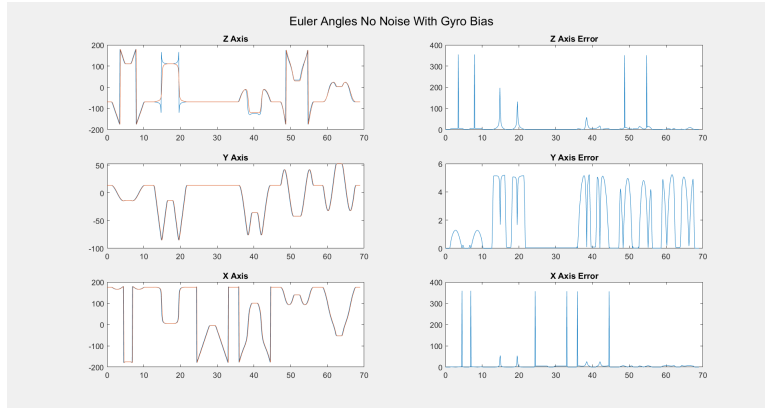


Figure 5: Euler Angles No Noise With Gyro Bias

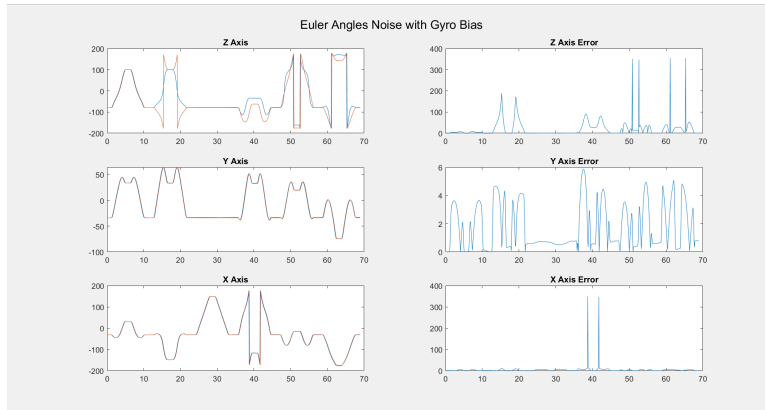


Figure 6: Euler Angles Noise with Gyro Bias



Figure 7: Euler Angles No Noise From Identity With Gyro Bias

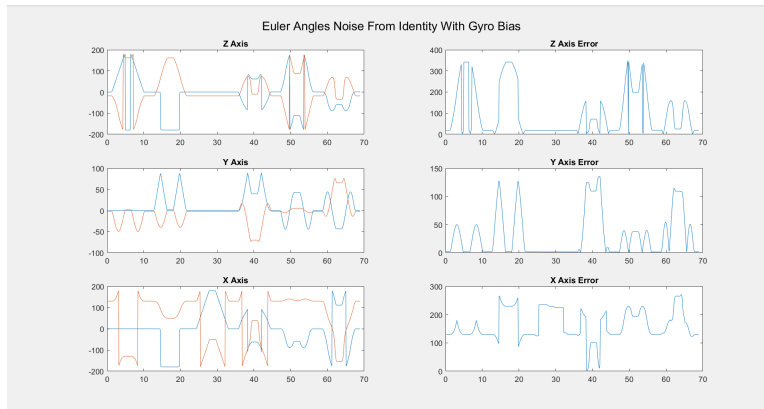


Figure 8: Euler Angles Noise From Identity With Gyro Bias

```
[Acc,Mag,wGyro,Eul] = CreateTrajectoryData(deltaT, 1);

testEul = [];
I = [1 0 0; 0 1 0; 0 0 1];
Rminus = I;
wGyro(1:1/deltaT,1);
wGyro(1:1/deltaT,2);
wGyro(1:1/deltaT,3);
Gyrobiasz = mean(wGyro(1:1/deltaT,1));
Gyrobiasy = mean(wGyro(1:1/deltaT,2));
Gyrobiasx = mean(wGyro(1:1/deltaT,3));
gyrobias = [Gyrobiasz,Gyrobiasy, Gyrobiasx ];

phi = deg2rad(Eul(1,3));
```

```

theta = deg2rad(Eul(1,2));
psi = deg2rad(Eul(1,1));
for i = 1:length(Eul)
    gyro = wGyro(i,1:3);
    gyro(1) = deg2rad(gyro(1) - gyrobias(1)) / 131 ;
    gyro(2) = deg2rad(gyro(2) - gyrobias(2)) / 131;
    gyro(3) = deg2rad(gyro(3)- gyrobias(3)) / 131;
    Rplus = IntegrateOpenLoop(Rminus, gyro, deltaT);
    theta = rad2deg(-asin(Rplus(1,3)));
    phi = rad2deg(atan2(Rplus(2,3), Rplus(3,3)));
    psi = rad2deg(atan2(Rplus(1,2),Rplus(1,1)));
    testEul(i, 1:3) = [psi, theta, phi];
    Rminus = Rplus;
end

figure;
subplot(3,2,1);
plot(t, testEul(:,1))
hold on;
plot(t,Eul(:,1))
title('Z Axis');

subplot(3,2,2);
plot(t, abs(testEul(:,1) - Eul(:,1)))
title('Z Axis Error');

subplot(3,2,3);
plot(t, testEul(:,2))
hold on;
plot(t,Eul(:,2))
title('Y Axis');
subplot(3,2,4);
plot(t, abs(testEul(:,2) - Eul(:,2)))
title('Y Axis Error');

subplot(3,2,5);
plot(t, testEul(:,3))
hold on;
plot(t,Eul(:,3))
title('X Axis');
subplot(3,2,6);
plot(t, abs(testEul(:,3) - Eul(:,3)))
title('X Axis Error');

sgtitle('Euler Angles Noise From Identity With Gyro Bias')

```

4.2 Real Data

To verify our matlab code we took 10 minutes of gyroscope data and used that as input for the gyroscope, starting with the identity matrix. What the desired outcome is the euler angles to be zero since there is no rotation. What occurred is that the euler angles would start at zero and slowly go up. This phenomenon is explained with the gyro bias drift that was shown in lab 3. Our results also showed that the y axis scales up/down much faster than the other axes which align with results from lab 3.

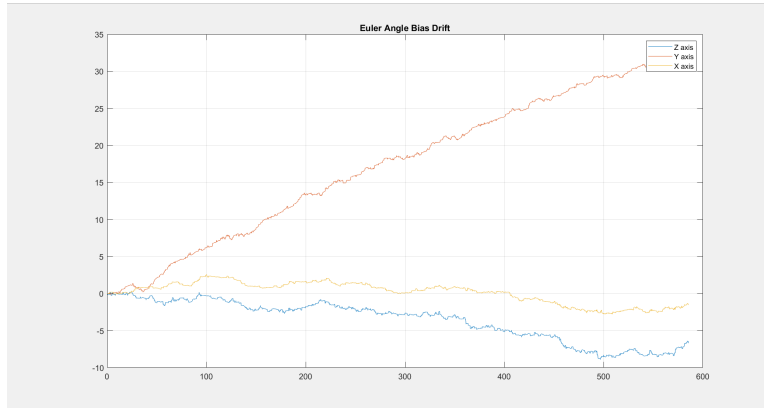


Figure 9: Euler Angles From Real Data Reflecting Bias Drift

When we recreated the same functionality into C it reflected the same pattern: the axes slowly going up/down due to bias drift, with the y axis scaling much faster than the other two.

```
#ifdef PART_4

int main(void)
{
    BOARD_Init();
    BNO055_Init();

    float r, q, p; // z, y, x
    float deltaT;

    deltaT = 0.1;

    float Rk[SIZE][SIZE] = {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}};
    float Rk1[SIZE][SIZE];
```

```

while (TRUE)
{
    r = ((BN0055_ReadGyroZ() - GYROBIASZ) * (M_PI / 180)) / 131;
    q = ((BN0055_ReadGyroY() - GYROBIASY) * (M_PI / 180)) / 131;
    p = ((BN0055_ReadGyroX() - GYROBIASX) * (M_PI / 180)) / 131;

    matrixOpenLoop(r, q, p, deltaT, Rk, Rk1);
    eulerAngles test = DCMtoEuler(Rk1);

    memcpy(Rk, Rk1, sizeof(Rk));

    printf("Roll: %f ", test.roll);
    printf("Pitch: %f ", test.pitch);
    printf("Yaw: %f\n", test.yaw);
}
}

#endif

void matrixOpenLoop(float r, float q, float p, float deltaT, float Rk[SIZE][SIZE], float Rout)

{
    float matrixExpOut[SIZE][SIZE];
    matrixExp(r, p, q, deltaT, matrixExpOut);

    multiplyMatrices(matrixExpOut, Rk, Rout);
}

t = 0:deltaT:(length(wGyro)-1)*deltaT;
Gyrobiasz = mean(wGyro(1:10/deltaT,3));
Gyrobiasy = mean(wGyro(1:10/deltaT,2));
Gyrobiasx = mean(wGyro(1:10/deltaT,1));
gyrobias = [ Gyrobiasx, Gyrobiasy, Gyrobiasz];

for i = 1:length(wGyro)
    gyro = wGyro(i,1:3);
    gyro(3) = deg2rad(gyro(3) - gyrobias(3)) /131;
    gyro(2) = deg2rad(gyro(2) - gyrobias(2)) /131;
    gyro(1) = deg2rad(gyro(1)- gyrobias(1)) /131;
    Rplus = IntegrateOpenLoop(Rminus, gyro, deltaT);
    theta = rad2deg(-asin(Rplus(1,3)));
    phi = rad2deg(atan2(Rplus(2,3), Rplus(3,3)));
    psi = rad2deg(atan2(Rplus(1,2),Rplus(1,1)));
    testEul(i, 1:3) = [psi, theta, phi];

```

```

        fprintf("psi: %f, theta: %f, phi: %f\n", psi, theta, phi);
        Rminus = Rplus;
    end
    testEul;
    figure
    plot(t, testEul(:,1))
    hold on;
    plot(t, testEul(:,2))
    hold on;
    plot(t, testEul(:,3))
    title('Euler Angle Bias Drift');
    grid on;
    legend('Z axis', 'Y axis', 'X axis');

```

5 Closed Loop Estimation

To simulate a closed loop estimation, we used the same procedure of part 4 of creating trajectory data and getting the resulting euler angles at each iteration. We used the `IntegrateClosedLoop` function in matlab which simulated a feedback control system based on the accelerometer and magnetometer which iteratively changed the bias and got each rotation matrix. The resulting euler angles resembled the various states a PD/ PID controller has for output.

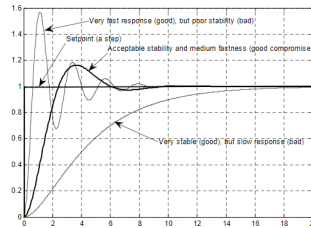


Figure 10: Feedback Response

```

clear all;
close all;
clc;
accelInertial=[0;0;-1];
magInertial=[1;0;0];
deltaT = .1;
[accels,mags,wGyro,Eul] = CreateTrajectoryData(deltaT, 1);
t = 0:deltaT:(length(wGyro)-1)*deltaT;
I = [1 0 0; 0 1 0; 0 0 1];
Rminus = I;
Gyrobiasz = mean(wGyro(1:1/deltaT,3));

```

```

Gyrobiasy = mean(wGyro(1:1/deltaT,2));
Gyrobiasx = mean(wGyro(1:1/deltaT,1));
Bminus = [Gyrobiasz, Gyrobiasy, Gyrobiasx];
for i = 1:length(wGyro)
    gyro = wGyro(i,1:3);
    gyro(1) = deg2rad(gyro(1) - Bminus(1)) / 131 ;
    gyro(2) = deg2rad(gyro(2) - Bminus(2)) / 131;
    gyro(3) = deg2rad(gyro(3)- Bminus(3)) / 131;
    [Rplus, Bplus] = IntegrateClosedLoop(Rminus, Bminus, gyro, mags, accels, magInertial, a);
    Bminus = Bplus;
    Rminus = Rplus;
    theta = rad2deg(-asin(Rplus(1,3)));
    phi = rad2deg(atan2(Rplus(2,3), Rplus(3,3)));
    psi = rad2deg(atan2(Rplus(1,2),Rplus(1,1)));
    testEul(i, 1:3) = [psi, theta, phi];
    fprintf("psi: %f, theta: %f, phi: %f\n", psi, theta, phi);
end
figure
plot(t, testEul(:,1),'b')
hold on;
plot(t, testEul(:,2),'r')
hold on;
plot(t, testEul(:,3),'g')
title('Euler Angle Closed Loop');
grid on;
legend('Z axis','Y axis', 'X axis');
%PrettyPlotAttitudeData(deltaT,accels,mags,wGyro,testEul)

```

5.1 Proportional Gain Feedback

5.1.1 Accelerometer

For the proportional gain feedback for the accelerometer, when K_{pa} was 1-70, it yielded a fast response with poor stability, when it was 75 it had acceptable stability with medium fastness, and a range of 77-100 was entirely unstable.

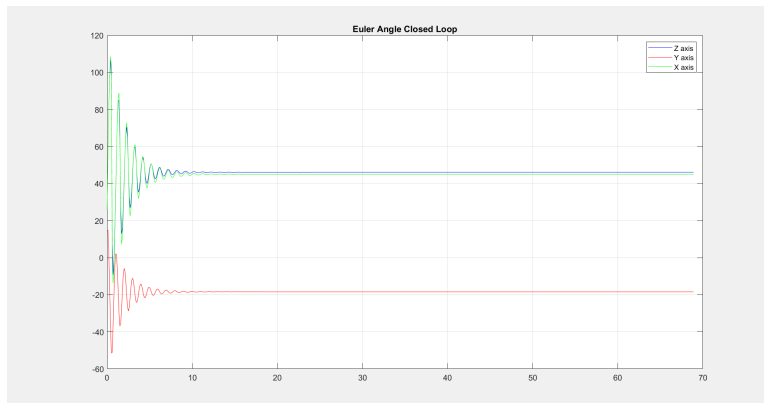


Figure 11: Fast Response, Poor Stability, $K_{pa} = 1$

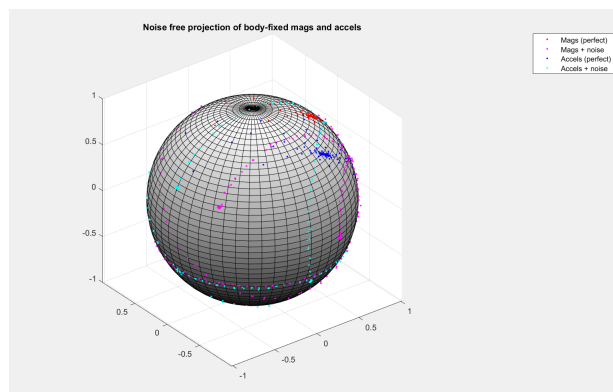


Figure 12: Fast Response, Poor Stability Projection

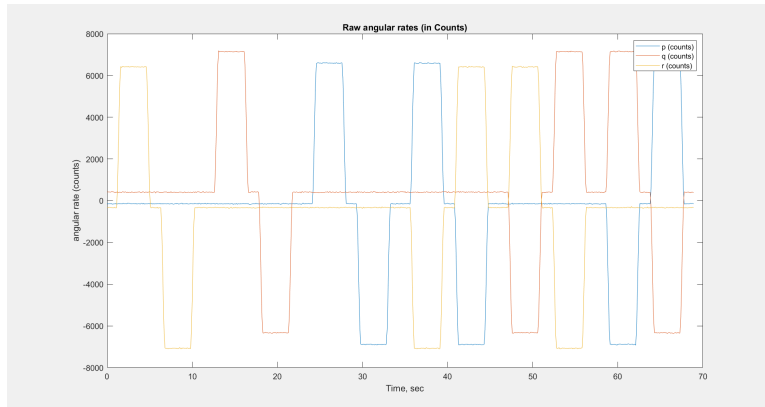


Figure 13: Fast Response, Poor Stability Angular Rates

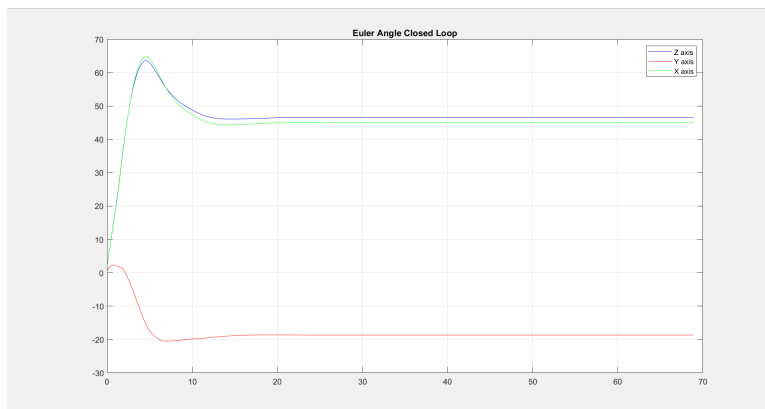


Figure 14: Acceptable Stability, Medium Fastness, $K_{pa} = 75$

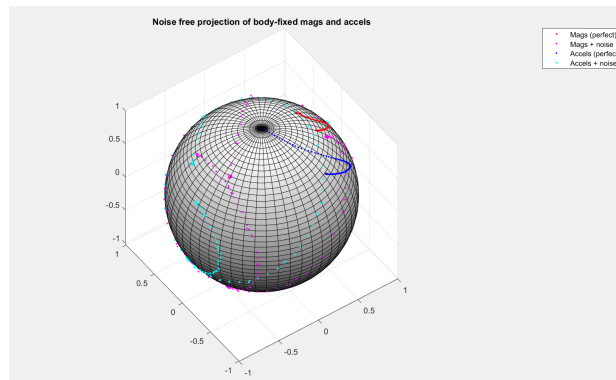


Figure 15: Acceptable Stability, Medium Fastness Projection

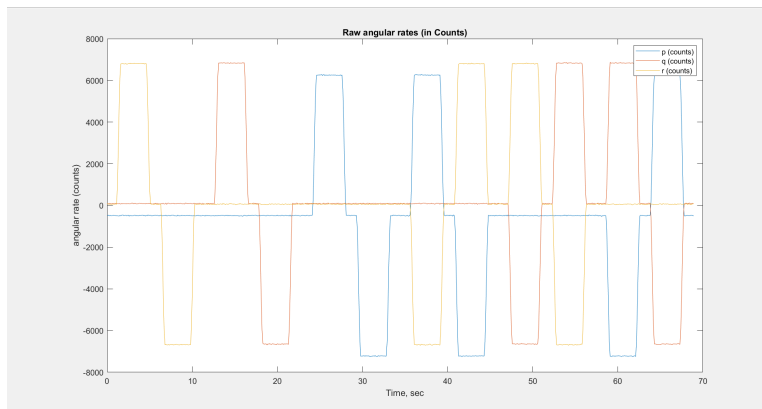


Figure 16: Acceptable Stability, Medium Fastness Angular Rates

80:100: unstable

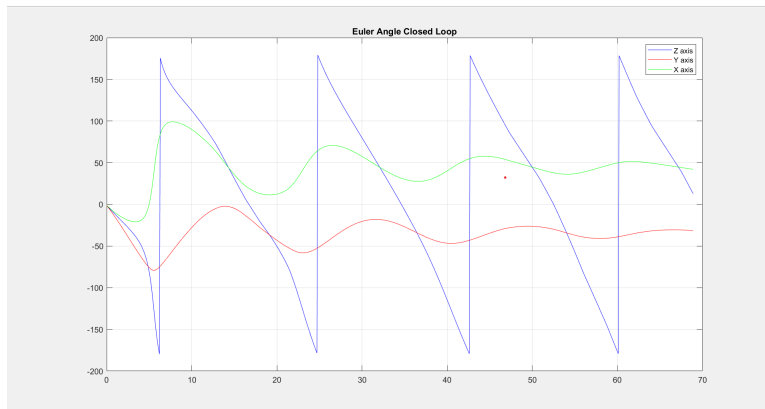


Figure 17: Unstable, $K_{pa} = 80$

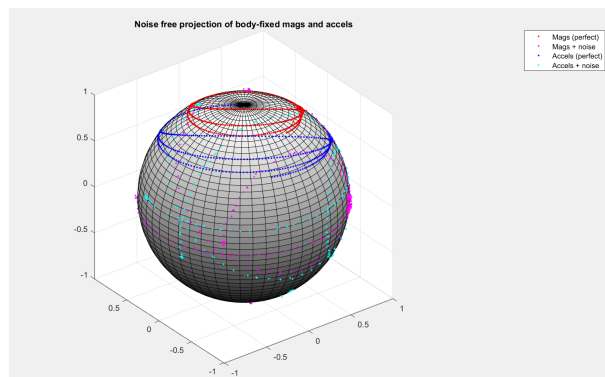


Figure 18: Unstable Projection

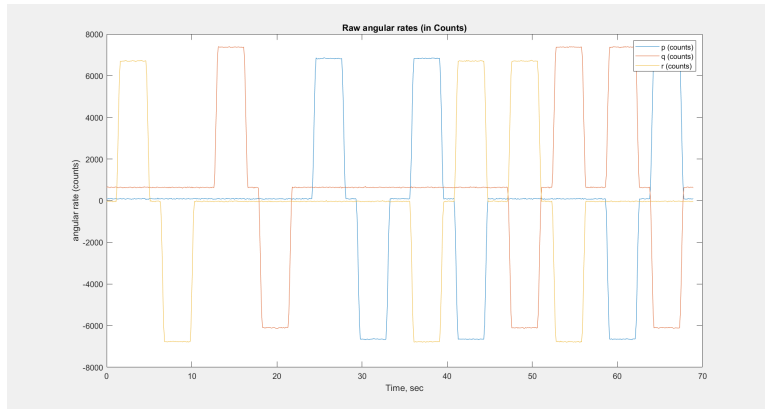


Figure 19: Unstable Angular Rates

5.1.2 Magnetometer

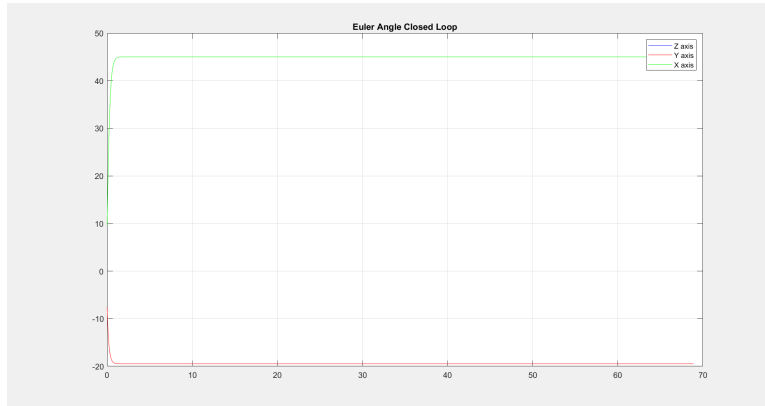


Figure 20: $K_{pm} = 5$

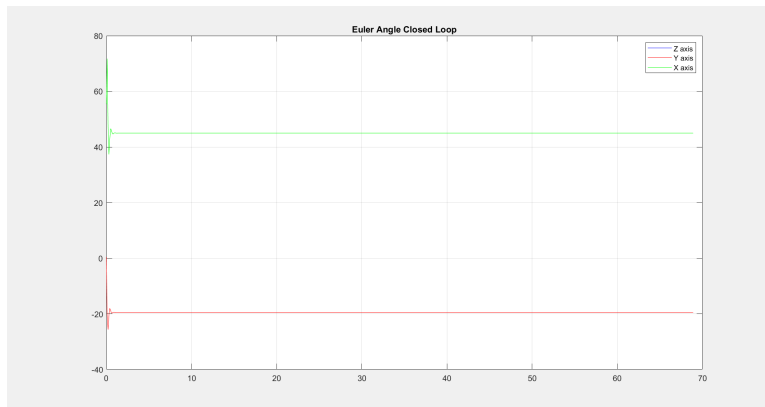


Figure 21: $K_{pm} = 10$

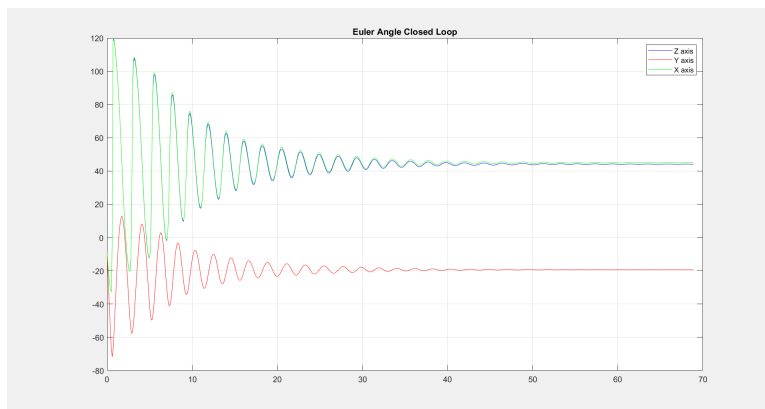


Figure 22: $K_{pm} = 25$

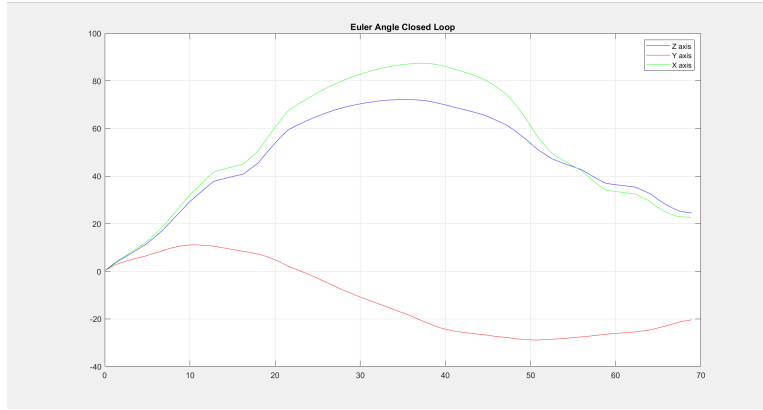


Figure 23: $K_{pm} = 50$

5.2 Proportional-Integral Gain Feedback

5.2.1 Accelerometer

For the proportional gain feedback for the accelerometer, when K_{pa} was 75 and K_{ia} is $K_{pa}/10$ it became a little faster and less stable from the medium stability medium fastness.

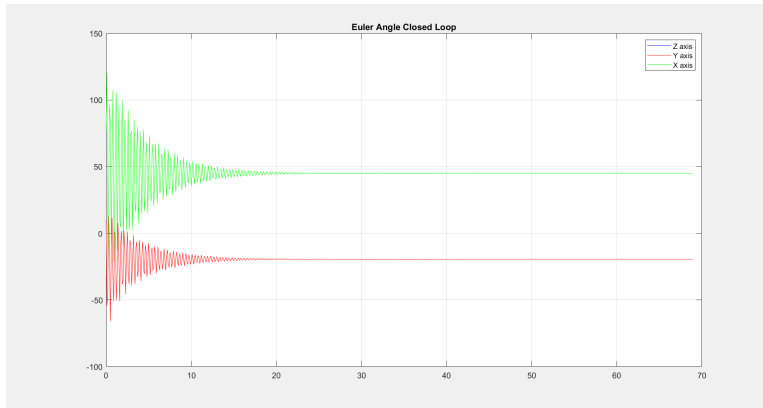


Figure 24: $K_{pa} = 1$, $K_{ia} = K_{pa}/10$

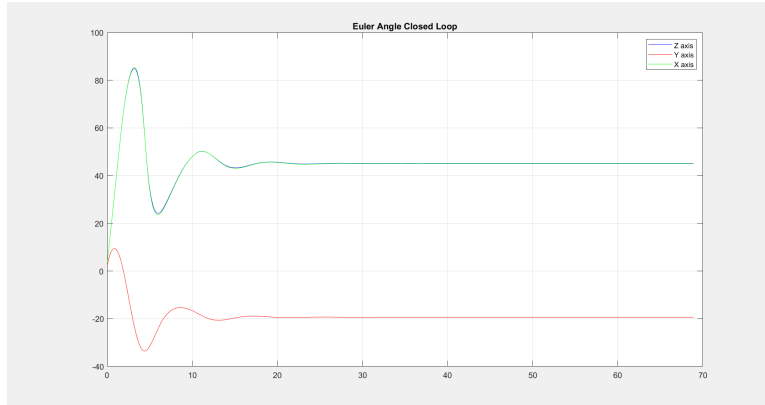


Figure 25: $K_{pa} = 75$, $K_{ia} = K_{pa}/10$

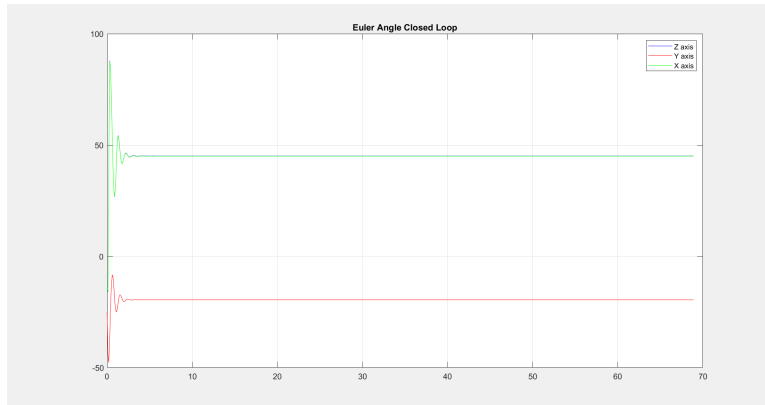


Figure 26: $K_{pa} = 80$, $K_{ia} = K_{pa}/10$

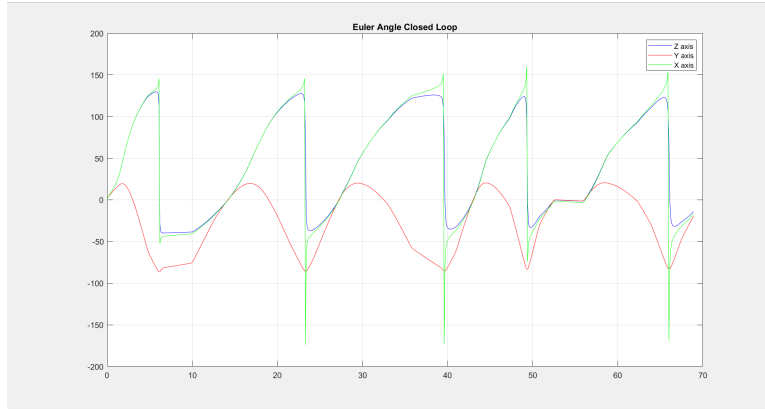


Figure 27: $K_{pa} = 1000$, $K_{ia} = K_{pa}/10$

5.2.2 Magnetometer

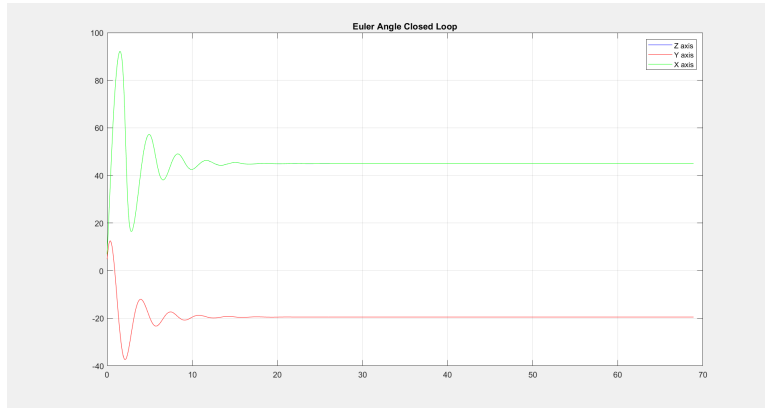


Figure 28: $K_{pm} = 1$, $K_{im} = K_{pm}/10$

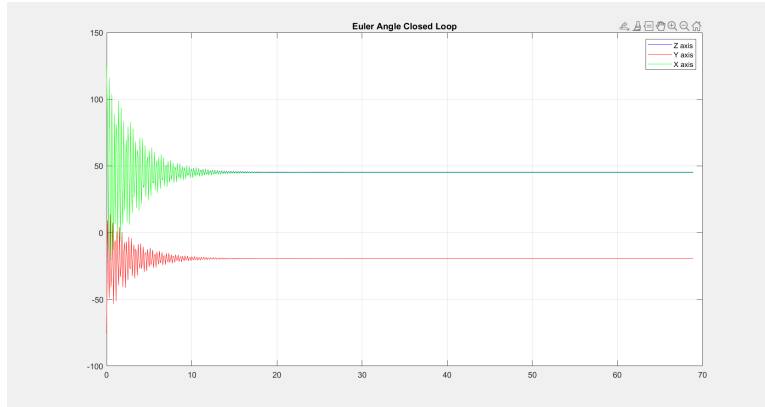


Figure 29: $K_{pm} = 5$, $K_{im} = K_{pm}/10$

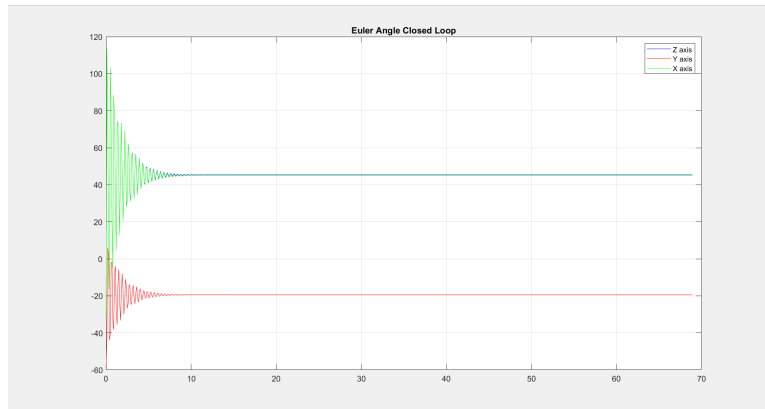


Figure 30: $K_{pm} = 10$, $K_{im} = K_{pm}/10$

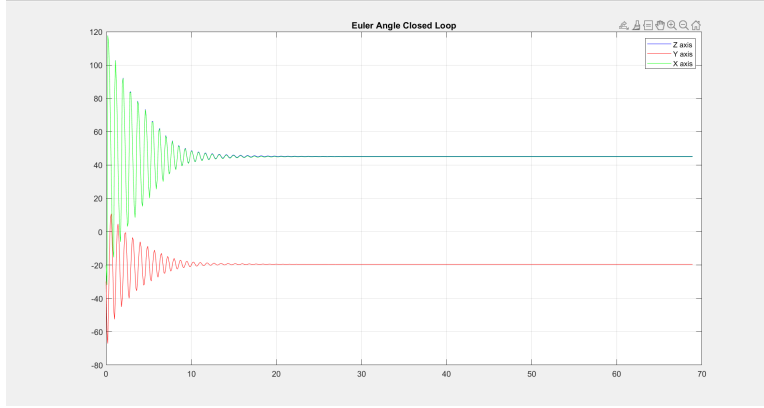


Figure 31: $K_{pm} = 25$, $K_{im} = K_{pm}/10$

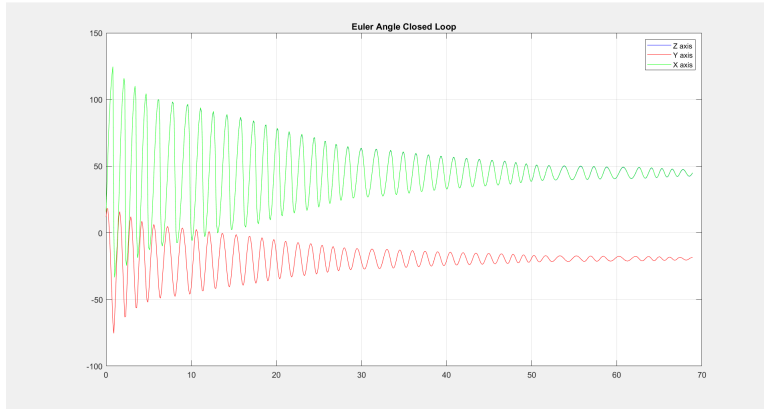


Figure 32: $K_{pm} = 50$, $K_{im} = K_{pm}/10$

6 Misalignment of Accelerometer and Magnetometer

To calculate the misalignment matrix we used the matlab code provided. We did a tumble test for the accelerometer and the magnetometer and used that data to pass into the function whabaSVD which computes the optimal rotation matrix to minimize the cost function of $J = 1/2 * \text{Sum}(w_i * |E_v - R * B_v|^2)$ using the SVD decomposition. This gave us the misalignment matrix. To test its accuracy, we multiplied the misalignment matrix R_{est} by the magnetometer data and used that as the magnetometer data into the whabaSVD function. This gave us small euler angles because the misalignment matrix became aligned with the magnetometer frame. The euler angles tell us how misaligned the matrix is, and a smaller value signifies that there is less misalignment.

```

%take a data
%take m data

a_m_data = [+00430, -00169, +00825, +00337, -00124, -00601;+00419, -00172, +00853, +00354, -

ax = a_m_data(:,1);
ay = a_m_data(:,2);
az = a_m_data(:,3);
mx = a_m_data(:,4);
my = a_m_data(:,5);
mz = a_m_data(:,6);

%plug into function [R_est, Pbody] = whabaSVD(B_v,E_v,w)
adata = [ax, ay, az]';
mdata = [mx, my, mz]';
[R_est, Pbody] = whabaSVD(adata,mdata);
%B_V body accelerometer
%E_V magnetometer
Rtest= R_est*mdata;
phi = rad2deg(atan2(R_est(2,3), R_est(3,3)))
theta = rad2deg(-asin(R_est(1,3)))
psi = rad2deg(atan2(R_est(1,2),R_est(1,1)))

%plug in a data to whabaSVD(B_v,E_v,w) where E_v = Rst * m data to check
[R_est, Pbody] = whabaSVD(adata,Rtest);
phi = rad2deg(atan2(R_est(2,3), R_est(3,3)))
theta = rad2deg(-asin(R_est(1,3)))
psi = rad2deg(atan2(R_est(1,2),R_est(1,1)))

%euler will be small in R_est

```

7 Full Complementary Attitude Estimation

For the last part of the lab we did a full complementary attitude estimation by combining the functionality of parts 5 and 6. We first got data from a new tumble test, doing spherical calibration for the accelerometer and the magnetometer but taking the gyro as raw measurements. We first corrected the magnetometer using the misalignment matrix and used that as input for the integration of the closed loop system. We took the calibrated accelerometer data, the calibrated and corrected magnetometer data, the raw gyroscope data and the bias from the mean of the first 10 seconds of the gyro readings to use our closed loop function to propagate euler angles at every time step. We messed around with K_{ia} , K_{pa} , K_{im} , K_{pm} to see how what the feedback response outputted.

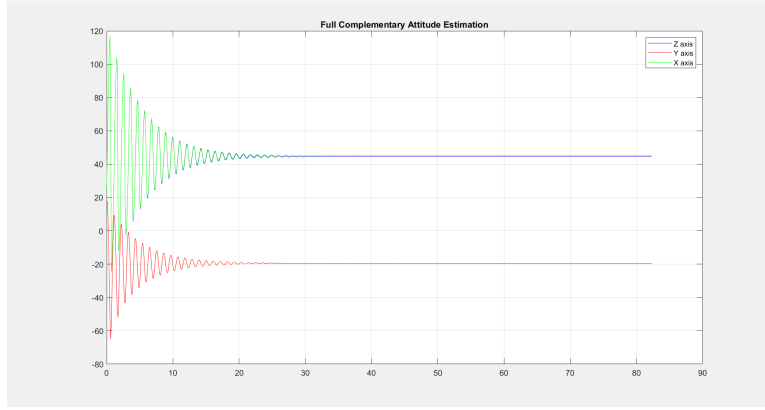


Figure 33: $K_{pa} = 1, K_{ia} = K_{pa}/10, K_{pm} = 1, K_{im} = K_{pm}/10$

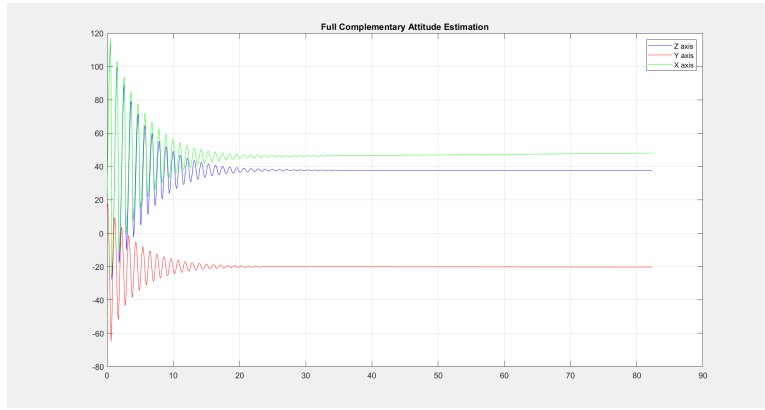


Figure 34: $K_{pa} = 20, K_{ia} = K_{pa}/10, K_{pm} = 20, K_{im} = K_{pm}/10$

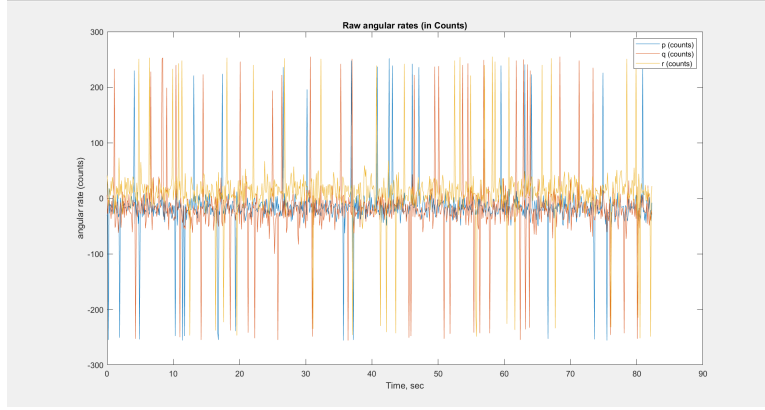


Figure 35: Angular Rates when Still

Making K_{pm} yielded a system that stabilized but had the x axis slope upward. It seems that increasing the K values distorts the slope of each of the axes.

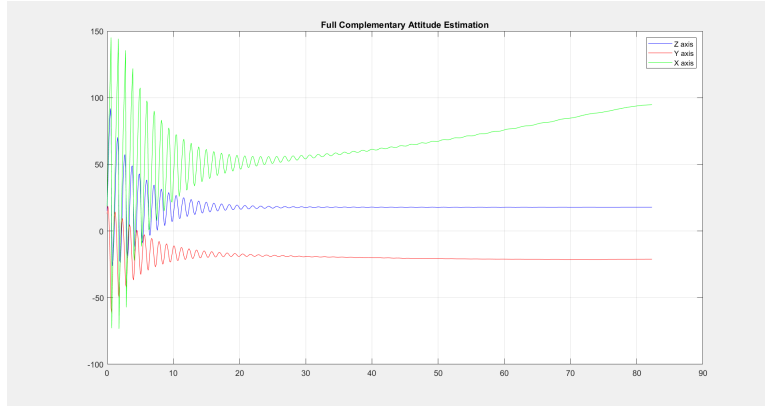


Figure 36: $K_{pa} = 75, K_{ia} = K_{pa}/10, K_{pm} = 150, K_{im} = K_{pm}/10$